



AI FULL-STACK & DEPLOYMENT ENGINEER · TRACK 2.B · SESSION 7

Provider Abstraction with LiteLLM & a Streaming Production UI

ClaimAssist v2: one OpenAI-compatible endpoint in, any provider out — routing, retries and failover behind the proxy, and a Next.js chat that streams tokens to the user

LiteLLM proxy · `model_list` & fallbacks · Multi-cloud (Azure · Bedrock · Vertex) · Vercel AI SDK · `useChat` + `streamText` · Failover demo

The lab runs entirely locally; a cloud key unlocks the optional failover path — same files, one uncommented block.

Day 7 · 2 hours · concepts 45 min + lab 60 min + review 15 min



SESSION 7 · AGENDA

Session agenda

45 min



Concepts & code walkthrough

Why provider abstraction is a business requirement; LiteLLM proxy, model_list, fallbacks, retries and cost-based routing; Azure OpenAI vs Bedrock vs Vertex AI; the Vercel AI SDK streaming architecture and time-to-first-token; route.ts line by line.

60 min



Lab — ClaimAssist v1 → v2

Start the three-service stack, verify the proxy, re-point the API with one env line, run the Next.js streaming chat, then stop the model container to observe retries, clean errors and (cloud path) automatic failover.

15 min



Review & wrap-up

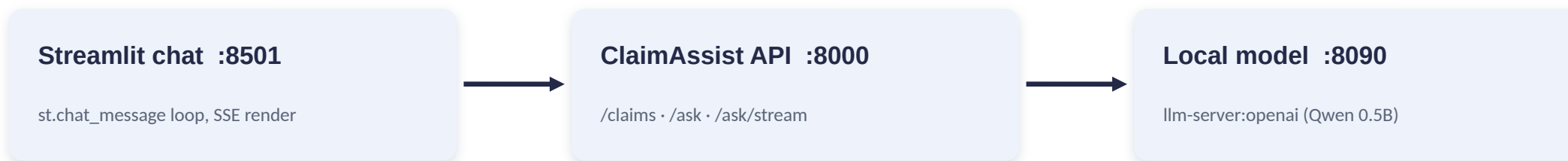
Compare failover evidence, revisit the local-to-production mapping, and preview Session 8: RAG with citations and multimodal intake.

Objective: make the model provider a configuration decision — swappable, fallback-protected and cost-aware — and ship a streaming chat UI on the production web stack.

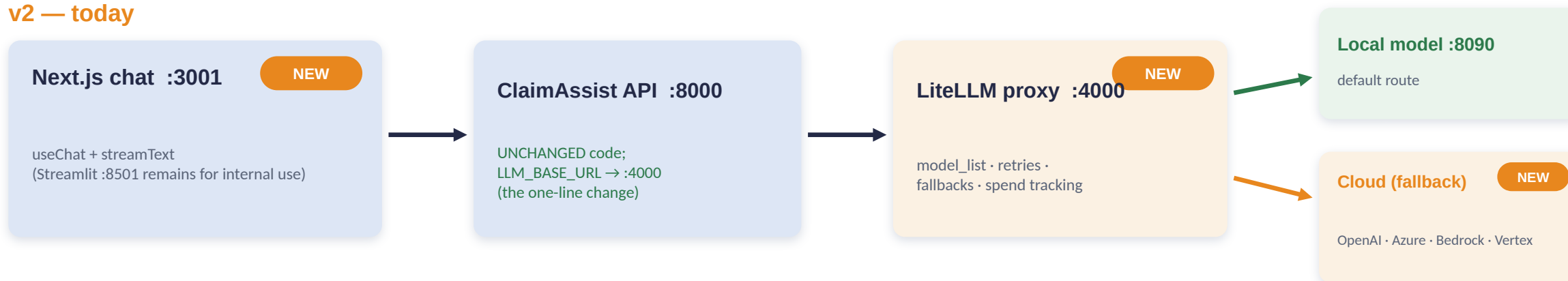


ClaimAssist v1 → v2: a proxy tier and a production UI

v1 — Session 6



v2 — today



The point of v2: every arrow still speaks the OpenAI protocol. The proxy tier changes no contract — it adds routing, resilience and cost governance behind an endpoint the application already knows how to call.



THE BUSINESS CASE

Why not hard-code one vendor: four forces



Quota exhaustion

Rate limits and token-per-minute quotas bind exactly at peak traffic. A second provider is surge capacity you can route to in one config line.



Provider outages

Every major provider has recorded multi-hour incidents. With one hard-coded vendor, their outage is your outage; with fallbacks, it is a routing event.



Cost differences

Per-token prices differ by 10-100× across models and change quarterly. Price-aware routing sends each request to the cheapest model that can serve it.



Model churn

Models are deprecated and replaced on the provider's timetable. When the model name is configuration, migration is an edit and an eval run — not a refactor.

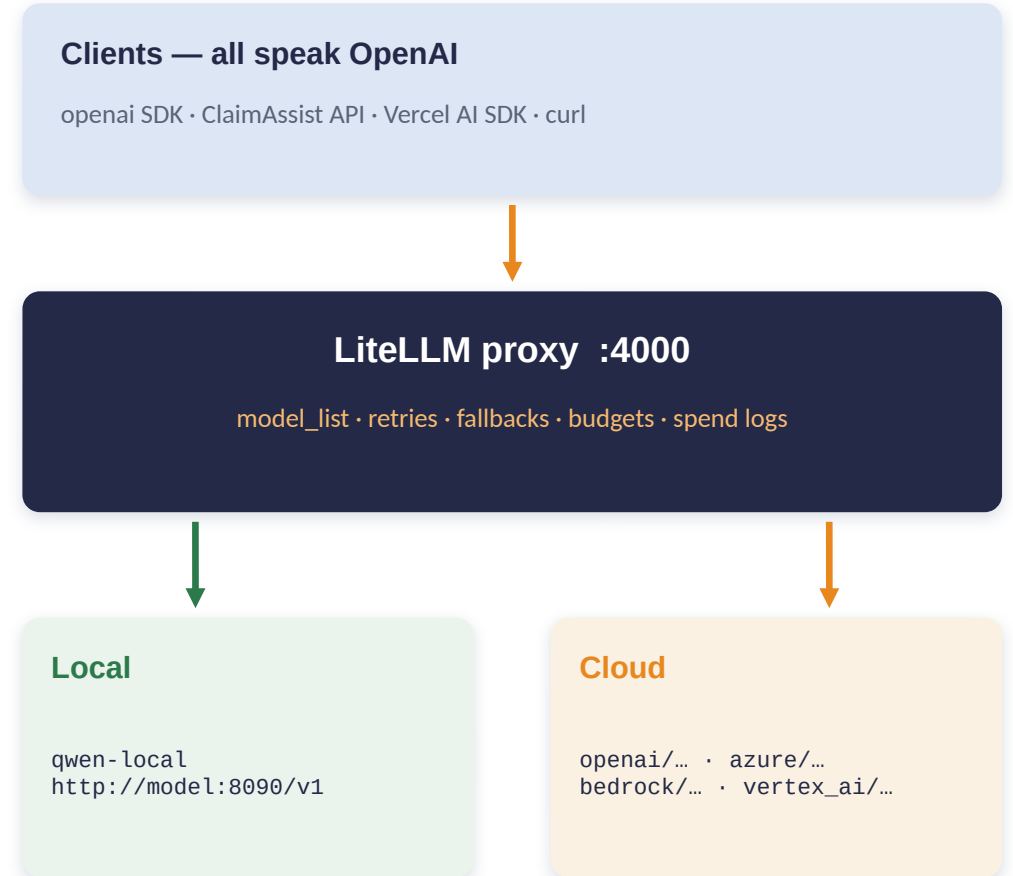
Architectural consequence: the provider is a routing decision made in configuration, behind one stable endpoint — never an SDK import repeated across the codebase.



LITELLM

LiteLLM: the OpenAI protocol in, any provider out

- LiteLLM translates the OpenAI chat-completions protocol to 100+ providers: OpenAI, Azure OpenAI, AWS Bedrock, GCP Vertex AI, Anthropic, Mistral, and any self-hosted OpenAI-compatible server — including our Qwen container.
- Two consumption modes: the Python SDK (litellm.completion, in-process) or the proxy server — a standalone gateway container. Today: the proxy, because it serves every client in every language and centralises keys, routing and spend.
- model_list maps a public model name ("qwen-local") to a deployment: provider, upstream model id, api_base, credentials. Clients only ever see the public name.
- The proxy's own API is the OpenAI protocol — so the openai SDK, the ClaimAssist API and the Vercel AI SDK all connect to it unmodified.



One sentence to retain: clients speak OpenAI to the proxy; the proxy speaks every provider's dialect upstream.



LITELLM

litellm/config.yaml — the actual lab file

```
# litellm/config.yaml — ClaimAssist v2 (abbreviated)
model_list:
  # LOCAL PATH — the Qwen shim is 'just another
  # openai provider' with a custom api_base
  - model_name: qwen-local
    litellm_params:
      model: openai/qwen-local
      api_base: http://model:8090/v1  # service name
      api_key: local

  # CLOUD PATH — uncomment one + set its env key
  # - model_name: gpt-4o-mini
  #   litellm_params:
  #     model: openai/gpt-4o-mini
  #     api_key: os.environ/OPENAI_API_KEY
  # ..azure/<deployment> . bedrock/anthropic.claude-3-haiku
  # ..vertex_ai/gemini-1.5-flash (see full file)

router_settings:
  fallbacks: [{"qwen-local": ["gpt-4o-mini"]}]]
  num_retries: 2
```

Reading the file

- `model_name` is the public name clients request; `litellm_params` is where it actually goes
- `os.environ/KEY` defers the secret to `.env` — no key ever sits in the YAML
- Enabling a cloud provider = uncomment + one key + docker compose restart litellm

router_settings

- `num_retries: 2` — retry the same deployment before declaring it failed
- `fallbacks` — then send the SAME request to the named alternates, in order
- Without a cloud entry the fallback finds no deployment: retries, then a clean error. That is the local half of Lab Step 4.



Routing policies: what the proxy decides for you

Policy	What it does	One config line
Retries	Re-attempt the same deployment on transient failure (timeout, 5xx, rate limit)	<code>num_retries: 2</code>
Fallbacks	Send the SAME request to ordered alternates — across models, providers, clouds	<code>fallbacks: [{"qwen-local": ["gpt-4o-mini"]}]</code>
Load balancing	Several deployments share one model_name; spread by latency, least-busy or round-robin	<code>routing_strategy: least-busy</code>
Cost-based routing	Pick the cheapest healthy deployment; prices from the bundled price map, or declared per entry	<code>routing_strategy: cost-based-routing</code>
Budgets & spend	Virtual keys with max_budget per team; every request costed and logged	<code>/key/generate · /spend/logs</code>

Order of events on failure: retry the deployment (num_retries) → fall back down the chain → only then return an error to the client.



Azure OpenAI vs AWS Bedrock vs GCP Vertex AI

	Azure OpenAI	AWS Bedrock	GCP Vertex AI
Auth model	API key per resource, or Microsoft Entra ID; endpoint is per-resource	AWS IAM — SigV4-signed requests via access key / role	Service account JSON / Application Default Credentials; OAuth2 tokens
Endpoint & naming	Your deployment name, not the model name: azure/<deployment>	Model id per region: bedrock/anthropic.claude-3-haiku-...	Project + location scoped: vertex_ai/gemini-1.5-flash
Regional availability	Model × region matrix; quota requested per region and per deployment	Availability differs per region; some models need region enablement	Location-bound (e.g. us-central1); model coverage varies by location
Model families	OpenAI models (GPT-4o family, o-series) under Azure terms	Anthropic Claude, Meta Llama, Amazon Titan/Nova, Mistral	Google Gemini plus partner models via Model Garden

Three providers, three unrelated auth schemes, three naming conventions — before a single token is generated.



What LiteLLM normalizes across providers

- **One request/response shape:** OpenAI chat-completions in and out, whatever dialect the provider speaks upstream.
- **Auth injection:** API key, SigV4 signing or OAuth2 service-account tokens handled per deployment — clients never carry provider credentials.
- **Streaming:** every provider's stream framing re-emitted as OpenAI SSE chunks.
- **Errors:** provider-specific throttling and fault codes mapped to one taxonomy that retries and fallbacks act on.
- **Usage & cost:** tokens counted and priced uniformly, powering spend logs and budgets.
- **Routing:** retries, fallbacks and load balancing operate across providers and clouds identically.

```
# the commented cloud entries in the lab file
# - model_name: gpt-4o-mini-azure
#   litellm_params:
#     model: azure/<your-deployment-name>
#     api_base: os.environ/AZURE_API_BASE
#     api_key: os.environ/AZURE_API_KEY

# - model_name: claude-3-haiku
#   litellm_params:
#     model: bedrock/anthropic.claude-3-haiku-...
#     aws_region_name: os.environ/AWS_REGION_NAME

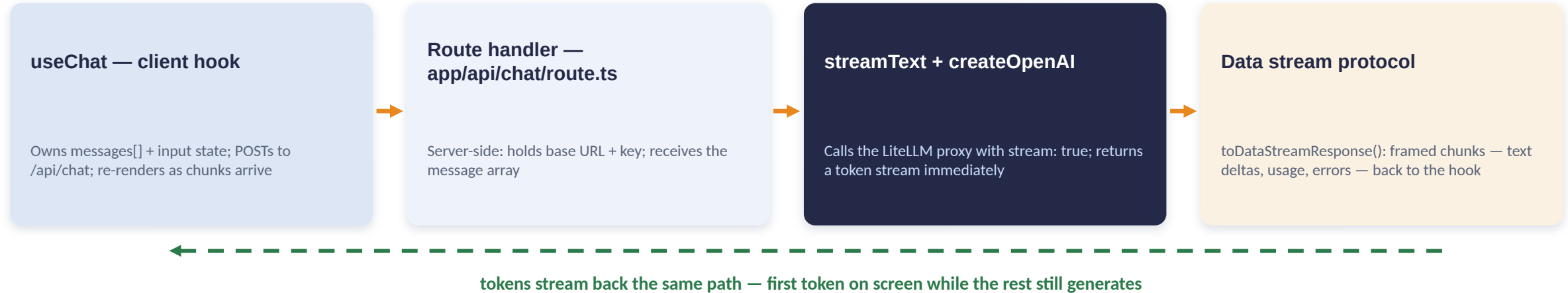
# - model_name: gemini-flash
#   litellm_params:
#     model: vertex_ai/gemini-1.5-flash
#     vertex_project: os.environ/VERTEXAI_PROJECT
#     vertex_location: os.environ/VERTEXAI_LOCATION
```

Enable any of these: uncomment + credentials in .env + restart the proxy. No app change.



STREAMING UI

Vercel AI SDK: the streaming architecture



Why a wire protocol at all?

- The data stream is framed: text deltas, tool calls, usage and errors each have a frame type
- useChat consumes it generically — the same hook renders plain chat today and tool-calling agents in Session 9
- Errors arrive as a frame, so the UI gets a typed error object, not a dropped connection

Where the secrets live

- **The browser talks ONLY to /api/chat on your own origin**
- Provider base URL and API key exist solely in the route handler's server environment
- A client-side call to a provider would ship the key to every visitor — the route handler is the secret boundary



Time-to-first-token: the UX metric that matters

Blocking response

~12 s

of spinner before the first visible word — the user cannot tell progress from failure.

Streaming response

<1 s

to the first token on screen; the same 12 s total, experienced as continuous progress.

What drives TTFT

- Queueing + prompt processing + first decode step
- Every network hop (browser → route → proxy → model) adds latency
- Fallback to a cloud model changes TTFT — monitor it per provider

Engineering rules

1. Stream by default for chat surfaces — total latency unchanged, perceived latency transformed.
2. Measure TTFT at the UI, where the user experiences it — not only at the model server.
3. Streaming needs error handling mid-stream: a connection that dies after token 50 must surface an error state, not a silently truncated answer.



app/api/chat/route.ts — the entire backend of the chat, line by line

```
import { streamText } from "ai";
import { createOpenAI } from "@ai-sdk/openai";

// Any OpenAI-compatible endpoint fits here — ours is
// the LiteLLM proxy. The key never leaves the server.
const litellm = createOpenAI({
  baseURL: process.env.LITELLM_URL ||
    "http://localhost:4000/v1",
  apiKey: process.env.LITELLM_API_KEY || "local",
});

export const maxDuration = 60; // slow CPU generations

export async function POST(req: Request) {
  const { messages } = await req.json();
  const result = await streamText({
    model: litellm(process.env.LLM_MODEL || "qwen-local"),
    system: "You are ClaimAssist, an insurance" +
      " claims-status assistant. ...",
    messages,
  });
  return result.toDataStreamResponse();
}
```

Line by line

- createOpenAI — a provider object for any OpenAI-protocol endpoint; baseURL = the proxy
- POST — the endpoint useChat calls; the hook sends the full message history
- streamText — starts generation and returns a live stream immediately
- toDataStreamResponse — frames tokens in the data stream protocol for useChat

What is deliberately absent

No SSE parsing, no provider SDK import, no key in the browser, no per-provider branches. Failover in Step 4 will happen without editing this file.



Streamlit vs Next.js: when each ships

	Streamlit (v1, :8501)	Next.js 15 + AI SDK (v2, :3001)
Time to working chat	Hours — one Python file, no build step	A day — components, route handler, styling
Audience	Internal: analysts, ops, demos, POCs	External: customers, at production polish
Streaming	Manual SSE loop over st.chat_message	First-class: useChat + data stream protocol
Auth, branding, UX control	Constrained by the framework	Full control — your design system, your auth
Deploy story	A container behind the VPN	Vercel or self-hosted Node/containers + CDN

The professional answer is sequencing, not either/or: validate with Streamlit in week one; ship customers Next.js. Both now sit on the same API and proxy.



HANDS-ON LAB · 60 MINUTES · LOCAL, WITH AN OPTIONAL CLOUD PATH

ClaimAssist v1 → v2

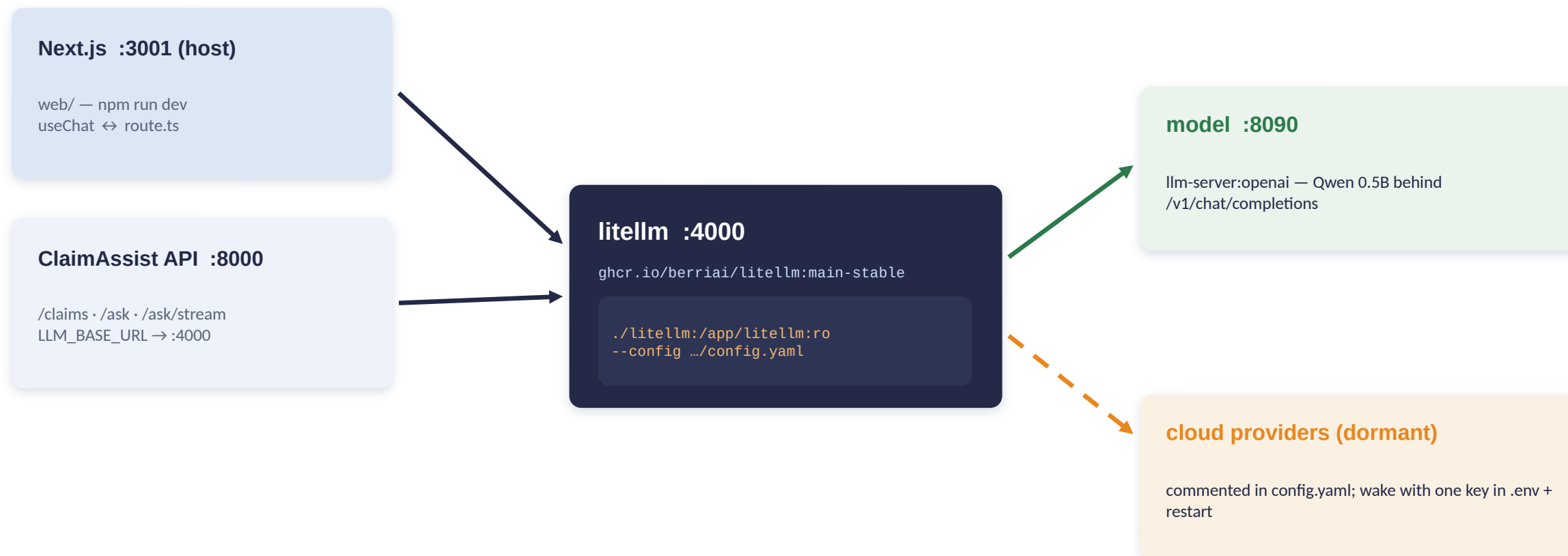
A LiteLLM proxy tier and a streaming Next.js chat

You will start the three-service stack, verify the proxy, re-point the API with one environment line, run the Next.js streaming chat, and then stop the model container mid-conversation to observe retries, clean errors — and, with a cloud key, automatic failover to another provider.

Step 0 setup · Step 1 stack + proxy check · Step 2 the one-line switch · Step 3 streaming UI · Step 4 resilience & failover · Step 5 cost & routing · Step 6 cleanup



The v2 stack: three containers plus a host-side web app



Two clients, one proxy: the API (server-to-server) and the Next.js route handler both call :4000 with the same protocol. The web app runs on the host — the normal frontend loop against containerised infrastructure.



LAB · STEP 0

Setup (8 min)

```
cd session7_lab

cp .env.example .env
# Windows PowerShell: Copy-Item .env.example .env

node --version      # expect v18 or newer

# install web deps now — ~2 min, overlaps Step 1
cd web && npm install && cd ..
```

What is in the handout

model_server/ and api/ (ClaimAssist v1, unchanged data and ports) · litellm/config.yaml · docker-compose.yml (model + litellm + api) · web/ (the Next.js 15 app, 7 files) · data/ (the six claims + three clause-numbered policy documents) · .env.example · README.md

Verification notes

- **Node.js 18+ is the only new prerequisite over Session 6 — verify it before anything else.**
- npm install takes roughly two minutes; start it now so it overlaps the compose build in Step 1.
- If npm reports a peer-dependency conflict, run `npm install --legacy-peer-deps` — the pinned versions are compatible.
- No cloud account is required for Steps 0-3 and the local half of Step 4; a single OpenAI key unlocks the failover variant.



Start the stack — and verify the PROXY first (12 min)

```
docker compose up -d --build
docker compose ps      # wait: model → healthy

# 1) the proxy lists its models
curl -s localhost:4000/v1/models | python -m json.tool
#   → one entry: "qwen-local"

# 2) a chat completion THROUGH the proxy
curl -s localhost:4000/v1/chat/completions \
  -H "Content-Type: application/json" \
  -d '{"model": "qwen-local", "messages":
    [{"role": "user", "content":
      "One sentence: what is a surveyor?"}]]}'
```

Checkpoint: a JSON answer arrives from port 4000. The proxy tier exists; nothing has used it yet.

Observe and interpret

- `/v1/models` is the proxy's public catalogue: **only uncommented `model_list` entries appear. Cloud models stay invisible until enabled.**
- The completion response is OpenAI-shaped JSON with usage counts — the client cannot tell a 0.5B local model answered rather than a cloud provider.
- First start downloads ~1 GB of model weights; watch docker compose logs -f model if health takes minutes.
- Ports in use? Session 6's stack may still be running — docker compose down in that directory first.



The one-line switch: the API now calls the proxy (8 min)

```
# docker-compose.yml – api service environment
# THE v1 -> v2 CHANGE: one env line.
#   v1: LLM_BASE_URL: http://model:8090/v1
LLM_BASE_URL: http://litellm:4000/v1
# api/app.py is byte-identical to Session 6
```

```
# structured answer (Pydantic contract)
curl -s localhost:8000/ask \
  -H "Content-Type: application/json" \
  -H "X-API-Key: local-dev-key" \
  -d '{"question": "What is the status of my
    claim?", "claim_id": "CLM-1001"}'
# → {answer, claim_id, status: "Under Review",
#   confidence: "high"}

# SSE stream – tokens as they arrive
curl -N localhost:8000/ask/stream ... \
  -d '{"question": "Why was my claim rejected?",
    "claim_id": "CLM-1003"}'
```

The teaching point

- This is why the course insisted on the OpenAI protocol as the contract in Session 6: infrastructure can now be inserted beneath the app without touching app code.
- The API keeps its own httpx retry loop for the single hop to the proxy; LiteLLM adds retries and fallbacks behind it — defence in depth, each layer guarding a different failure.
- Both endpoints respond exactly as in v1. The upgrade is invisible at the contract — which is the definition of a good abstraction.



The Next.js streaming chat (12 min)

```
cd web
npm run dev          # → http://localhost:3001

# ask: "What does the motor policy say
#      about licence validity?"
```

The chain each keystroke travels

```
useChat (page.tsx) → POST /api/chat (route.ts)

→ streamText + createOpenAI → LiteLLM :4000

→ model :8090 — tokens stream back the same path
```

Checkpoint: the first words appear well before the answer finishes — you are watching time-to-first-token beat total latency in real time.

Read the code as it runs

- **page.tsx:** no fetch call, no SSE parser — useChat owns the request, the message array and the re-render per chunk.
- **route.ts:** the base URL and key live here, server-side. View page source in the browser: no key, no provider URL anywhere.
- The web chat answers from the model's general knowledge plus the policy phrasing in your question; the API grounds answers in claim records. Which surface should own grounding is a Session 8 question — RAG.
- Keep this chat open — Step 4 breaks it on purpose.



Resilience, local version: kill the model (6 min)

```
docker compose stop model    # simulated outage

# ask another question in the chat at :3001
# → LiteLLM retries (num_retries: 2)
# → no fallback deployment enabled
# → clean, typed error in the UI error box

docker compose start model   # recover; chat again
```

What did NOT happen

- No infinite spinner — the failure is reported, not hidden
- No crashed page — the error arrives as a data-stream frame, and the UI has a rendering path for it
- No silently truncated answer presented as complete

Observe and interpret

- The failure sequence is deliberate: retry the deployment, walk the fallback chain, find nothing enabled, return a structured error. Every stage is configuration you wrote.
- The UI's error box is not cosmetic: streaming interfaces need an explicit error state, or a dead upstream looks identical to a slow one.
- This is the guaranteed floor: even with zero cloud keys, an outage degrades to a clean, user-readable failure in seconds — not a hang.
- Keep the chat open; the second half upgrades this exact failure into a non-event.



The marquee demo: automatic cloud failover

```
# litellm/config.yaml: uncomment gpt-4o-mini
# .env: OPENAI_API_KEY=sk-...
docker compose restart litellm

docker compose stop model
# ask again at :3001 ...
# → the CLOUD model answers. Same chat,
#   same code, different provider.
```

Why this is the marquee demo

- A provider outage became a routing event: the user saw a slower answer, not an error.
- Not one line of application code changed between failure and failover — only configuration.
- The identical mechanism chains provider→provider across Azure, Bedrock and Vertex: multi-cloud resilience is this YAML, extended.

1 · Request

useChat → route.ts → proxy :4000

2 · Local attempt

qwen-local unreachable — model stopped

3 · Retries

num_retries: 2 — both fail

4 · Fallback

chain names gpt-4o-mini — enabled now

5 · Cloud answers

tokens stream into the SAME chat

Afterwards: docker compose start model — traffic returns to the local (cheapest) deployment. Note the fallback model differs in capability and cost: that observation feeds the pitfalls discussion.



Cost & routing discussion, then cleanup (8 min)

- **Step 5 — read the comment block at the bottom of `litellm/config.yaml`, then discuss:**
- `routing_strategy`: cost-based-routing — with several deployments behind one `model_name`, the router picks the cheapest healthy one per request.
- Where prices come from: LiteLLM's bundled price map (`model_prices_and_context_window.json`); self-hosted entries declare `input_cost_per_token` / `output_cost_per_token` themselves.
- Budgets & spend: run the proxy with a database + `LITELLM_MASTER_KEY`, issue virtual keys with `max_budget` via `/key/generate`, read `/spend/logs` per key and team.
- Discussion prompt: which ClaimAssist requests justify a frontier-model price, and which should never leave the local deployment?

```
# excerpt — litellm/config.yaml
# router_settings:
#   routing_strategy: cost-based-routing
#
# self-hosted economics, per entry:
#   litellm_params:
#     input_cost_per_token: 0.0000004
#     output_cost_per_token: 0.0000012
```

```
# Step 6 — cleanup
docker compose down
# keep hf_cache (model weights) for S8:
docker volume ls | grep hf_cache
```

Keep the directory — Session 8 adds RAG on this stack.



SYNTHESIS

Mapping the local lab to production

Local mechanism — today's lab	Production counterpart
<code>LiteLLM proxy container on :4000</code>	The LLM gateway / API-gateway tier fronting all model traffic
<code>config.yaml fallbacks + num_retries</code>	Multi-provider / multi-region resilience policy as reviewed infrastructure-as-code
<code>Commented cloud entries + keys in .env</code>	A secrets manager injecting provider credentials into the gateway only
<code>cost-based-routing + price map discussion</code>	LLM FinOps: price-aware routing, budgets, per-team spend reporting
<code>next dev -p 3001 on the host</code>	The same app on Vercel or self-hosted Node containers behind a CDN
<code>route.ts holding base URL + key</code>	The server-side secret boundary — provider keys never reach the browser

The proxy pattern, the fallback policy and the secret boundary transfer unchanged; only the hosting differs.



LAB DELIVERABLES

Evidence to submit at the end of the session

1

Proxy evidence

A screenshot of `curl localhost:4000/v1/models` listing `qwen-local` — the proxy catalogue with only the local deployment enabled (Step 1).

2

Streaming evidence

A screenshot of the Next.js chat at `:3001` captured mid-stream — a partially rendered answer visibly growing (Step 3).

3

Failover / resilience evidence

Either the cloud-path answer arriving while the model container is stopped, or the local path's retry-then-clean-error in the UI (Step 4).

4

Written answer

A short paragraph: when is multi-provider routing worth its operational complexity — and when is one provider with good error handling enough?

Submission closes the lab block; the review compares failover evidence and the written answers across pairs.



PITFALLS

Common failure modes in provider abstraction and streaming UIs



Hard-coding one provider SDK

Provider imports scattered through the codebase recreate vendor lock-in. Keep the OpenAI protocol at the boundary and route through the gateway — swapping vendors stays a config change.



Silent fallback to a different model

A fallback model differs in capability and price. Log every fallback event and alert on it — an outage that quietly degrades answer quality is still an incident.



No budget alerts on cloud keys

A persistent failover to a paid model converts an outage into an invoice. Set max_budget on virtual keys and alert on spend velocity from day one.



Streaming without error boundaries

A stream that dies mid-answer must surface an error state. Without one, a truncated answer is indistinguishable from a complete one — the worst failure mode.



Exposing provider keys to the browser

Any client-side provider call or NEXT_PUBLIC_ key ships the credential to every visitor. Keys live in the route handler and the proxy — nowhere else.



Ignoring cost deltas between models

Fallback and load-balanced models can differ 10-100× in per-token price. Cost per answered request is a monitored metric, not an end-of-month surprise.



SESSION 7 · RECAP

Capabilities established today



Abstract the provider

The model vendor is a config entry behind a LiteLLM gateway — never an import statement.



Route with policy

Retries, ordered fallbacks, load balancing and cost-based routing declared in one YAML.



Survive an outage

A stopped model degrades to a clean error locally — or a cloud failover with a key. Zero code changes.



Stream to production

useChat ↔ route.ts ↔ streamText: tokens on screen at TTFT, errors as typed frames.



Govern the spend

Price-aware routing, virtual keys with budgets, per-request spend logs at the proxy.

Next session

RAG with Citations + Multimodal Intake

ClaimAssist v3: the policy documents are chunked by clause and retrieved per question, so answers cite exact clause ids — [M-2.3], [H-4.2] — with an honest NOT_IN_POLICY state; and an intake endpoint extracts structured fields from uploaded claim documents.

Retain the session7_lab stack — Session 8 builds RAG on this same proxy and model.